

Fundamentos de los Sistemas Operativos

Tema 2. Procesos

© 1998-2024 José Miguel Santos - Alexis Quesada - Francisco Santana

Contenidos del Tema 2

- Qué es un proceso
- Estructuras de datos para gestionar procesos
- API para trabajar con procesos
- Hilos (*threads*)
- Algoritmos de planificación de la CPU

Bibliografía para el Tema 2

- Silberschatz, 10th edition:
 - **Chapter 3 (processes)**
3.1, 3.2, 3.3
 - **Chapter 4 (threads and concurrency)**
4.1, 4.2, 4.3, 4.4
 - **Chapter 5 (CPU scheduling)**
entero, excepto 5.6

¿Qué es un proceso?

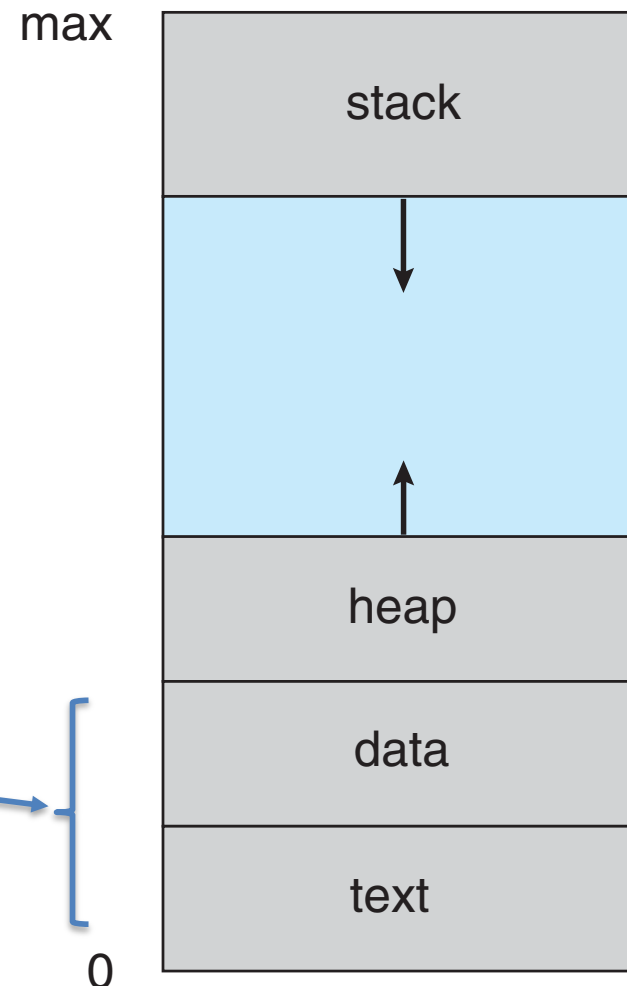
- Un **proceso** es un programa en ejecución.
- Un proceso nuevo se crea a través de una llamada al sistema (Windows: `CreateProcess`; Unix: `fork`, `execve`).
- Un proceso necesita recursos para poder ejecutarse: memoria RAM, dispositivos de E/S, la CPU, etc.
- Casi siempre el proceso tendrá que competir con otros procesos para obtener tiempo de CPU y áreas de memoria.

Organización de la memoria interna de un proceso (Unix, Windows)

Zonas típicas de la memoria:

- código (llamado «texto»)
- datos (variables globales + memoria dinámica)
- pila (*stack*) (datos locales de subrutinas)

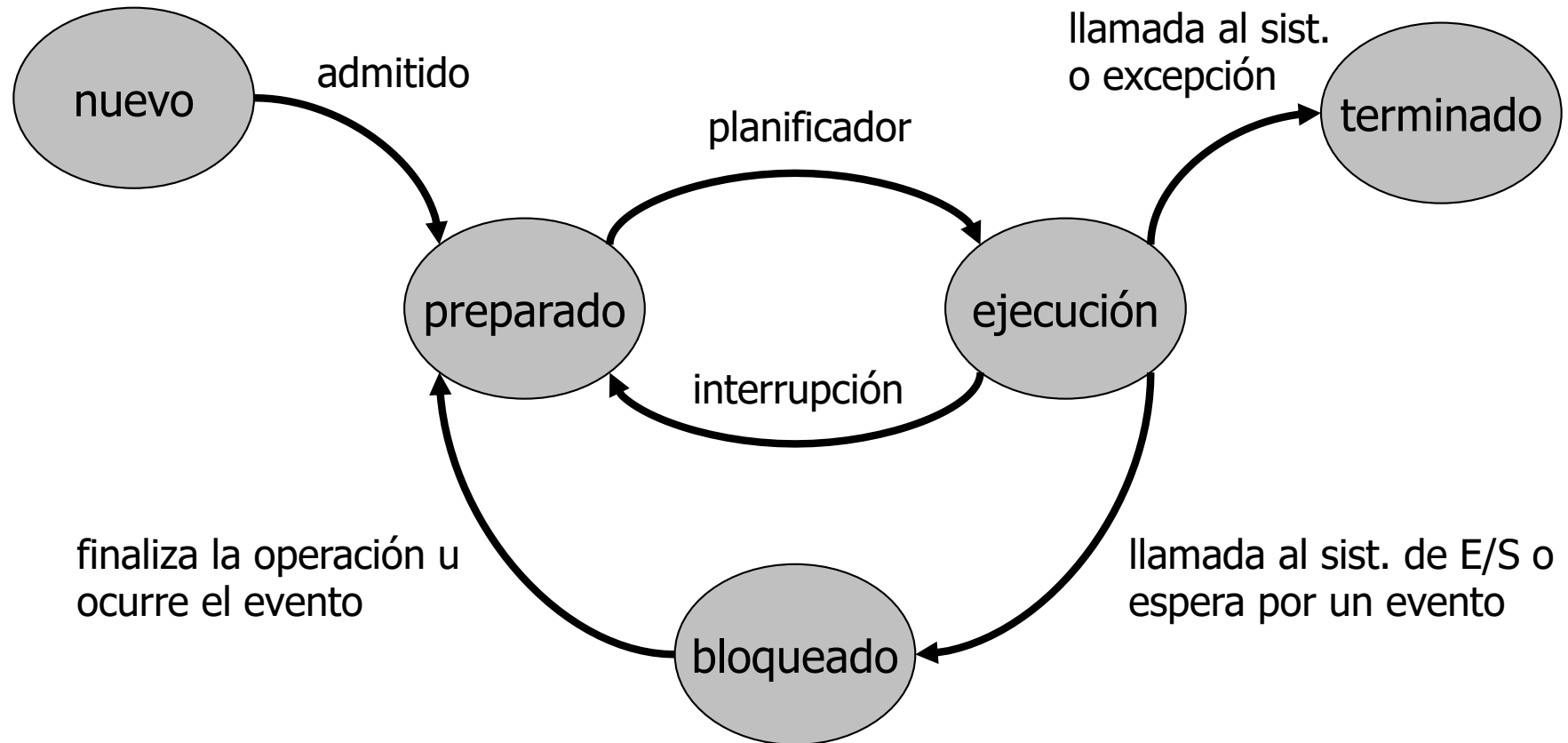
Las zonas «text» y «data» se copian desde el fichero ejecutable



Estados de un proceso

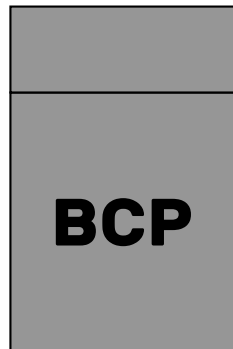
- A medida que un proceso se ejecuta, cambia de estado:
 - **nuevo**: recién creado por el S.O.
 - **en ejecución**: está en la CPU ejecutando instrucciones
 - **bloqueado**: esperando a que ocurra algún evento (ej. una operación de E/S)
 - **preparado**: esperando a que le asignen un procesador
 - **terminado**: no ejecutará más instrucciones y el S.O. le retirará los recursos que consume

Estados de un proceso



Bloque de control de proceso (BCP)

Cada proceso tiene asociada una estructura de datos que almacena su estado y otra información de control:



- Valores de los registros de la CPU
- Estado actual (preparado, bloqueado, etc.)
- Información para el planificador (prioridad, tiempo de espera...)
- apuntadores a los recursos de memoria, E/S, archivos etc. que tiene concedidos o abiertos
- Información de contabilidad (tiempo consumido...)

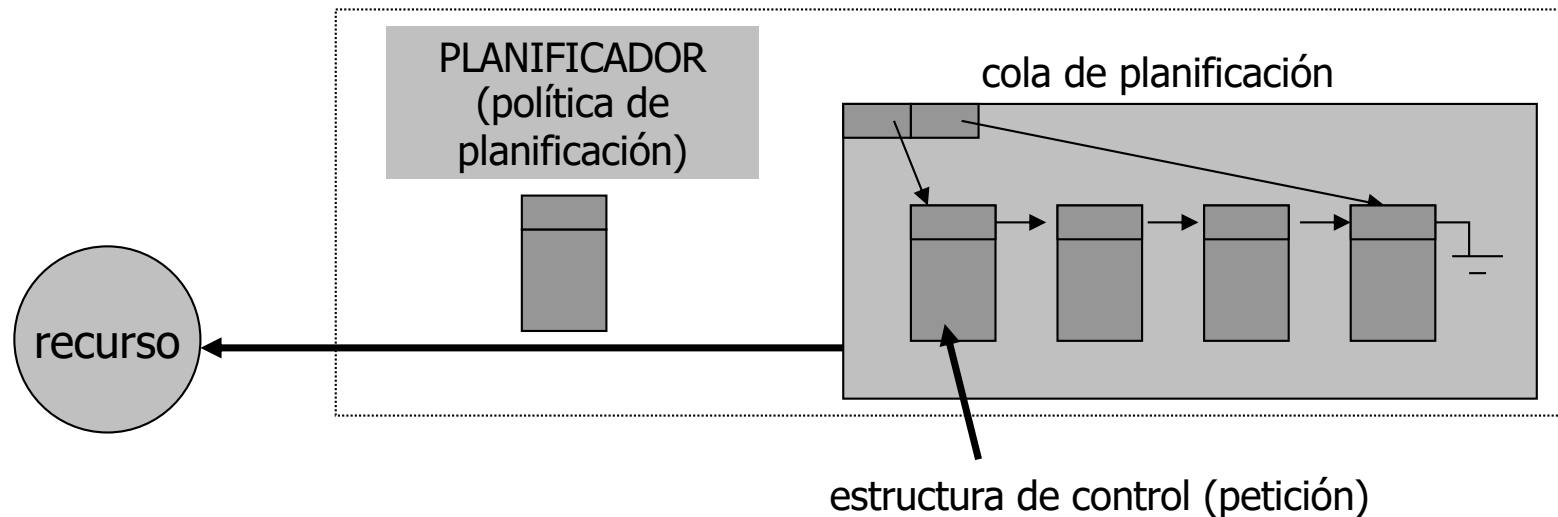
Planificación de la CPU

(CPU scheduling)

- ¿Qué proceso utiliza la CPU en cada momento?
- Objetivos diferentes según el tipo de sistema:
 - Sistemas por lotes → sacar rendimiento al procesador (*throughput*)
 - Sistemas interactivos → garantizar tiempos de respuesta cortos (*responsiveness*)
 - Tiempo real → garantizar plazos de ejecución (*meet deadlines*)

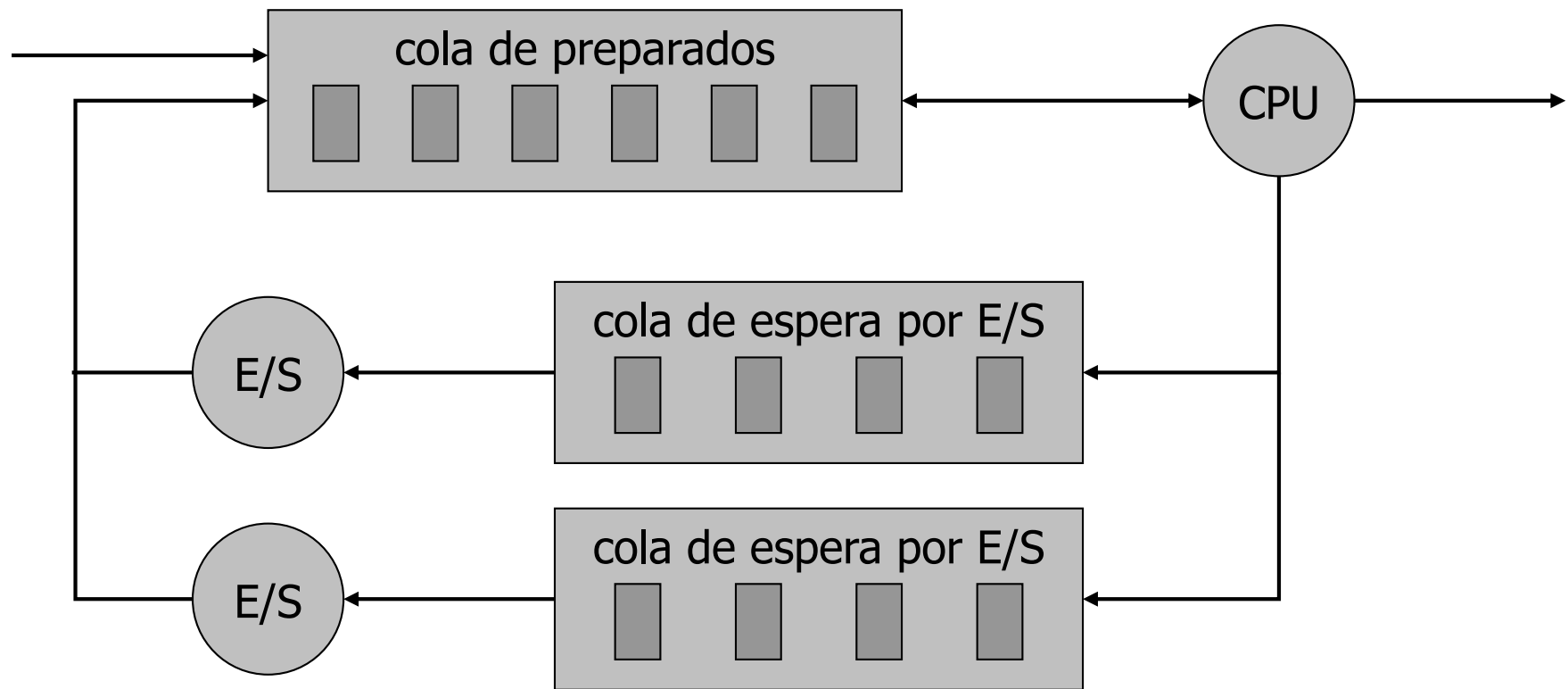
Cola de planificación

- Conjunto de procesos en espera por la utilización de un determinado recurso
- Implementación: tabla, lista enlazada, árbol...
- El planificador inserta y extrae procesos de la cola de acuerdo con la política implementada



Colas de procesos

- El SO organiza los BCP en colas de espera por el procesador o por los dispositivos de E/S. (colas de planificación: cola de procesos, colas de dispositivos)

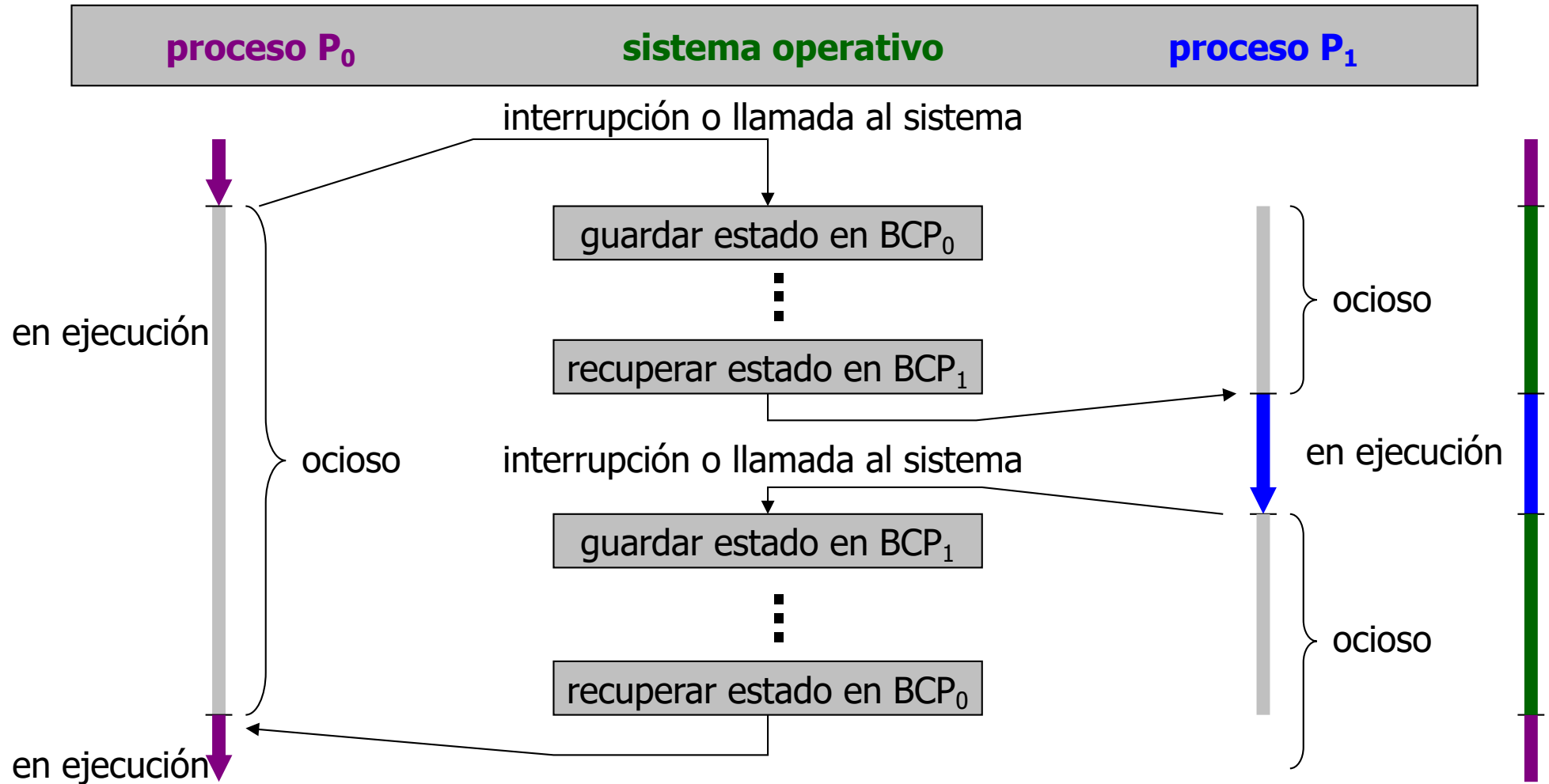


Cambio de contexto

(context switch)

- Es la operación que consiste en desalojar a un proceso de la CPU y reanudar otro.
- El módulo del SO encargado de esta acción se suele llamar **despachador** (*dispatcher*).
- Hay que guardar el estado del proceso que sale en su BCP, y recuperar los registros del proceso que entra. Cada contexto se encuentra en cada BCP.
- El cambio de contexto es *tiempo perdido*, así que debe ser lo más rápido posible.
- El hardware a veces ofrece mecanismos para reducir el tiempo del cambio de contexto (ej. en x86, instrucción PUSH/POP = guarda/recupera en la pila todos los registros; en ARM, instrucciones LDM/STM).

Cambio de contexto



API PARA TRABAJAR CON PROCESOS

¿Qué ofrece una API de procesos?

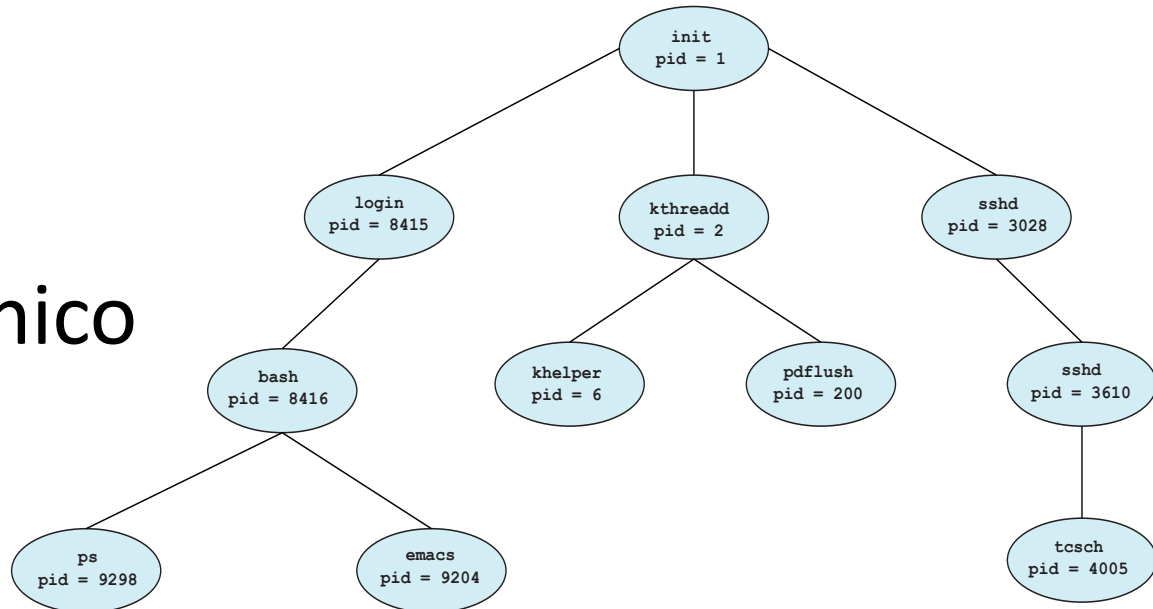
- Creación de nuevos procesos
- Finalización de procesos
- Comunicación entre procesos (IPC)

Creación de procesos

- Todo proceso se crea mediante una llamada al sistema ejecutada por otro proceso.
- Al creador se le llama **proceso padre** y al nuevo, **proceso hijo**.
- Relación entre el padre y el hijo:
 - ¿ejecutan el mismo código?
 - ¿utilizan la misma área de memoria?
 - ¿comparten archivos abiertos y otros recursos?
 - cuando el padre termina, ¿el hijo muere con él?

Identificadores y árboles de procesos

- Cada proceso recibe un identificador único (**PID** en Unix y Windows)
- Los padres e hijos van formando un **árbol de procesos**



Ejemplos de llamadas al sistema

- Windows:
 - **CreateProcess()**: se indica el fichero ejecutable donde está el código del hijo
- UNIX:
 - **fork()**: crea un proceso hijo que es un duplicado del padre
 - **exec()**: sustituye el código por un nuevo fichero ejecutable (no crea un nuevo proceso)

Ejemplo de fork() y exec()

```
pid_t pid = fork();
if ( pid!=0 ) {
    puts ("Soy el proceso padre");
    // otras acciones del proceso padre
}
else { // el hijo entrará por aquí
    puts ("Soy el proceso hijo");
    // el hijo ejecuta la orden "ls -l"
    // que reemplaza el código actual del hijo
    execvp ("ls", "ls", "-l", NULL);
}
```

Terminación de procesos

- Un proceso termina cuando invoca a una llamada al sistema específica, ej. **exit()**
- También si se genera una excepción y el S.O. decide abortarlo.
- En UNIX, cuando un proceso termina, con él mueren sus descendientes.
- Podría existir una llamada al sistema para abortar otro proceso, ej. **kill()**
- Esperar por la terminación de un proceso hijo: **wait()**

Ejemplo de sincronización con wait()

```
pid_t pid = fork();
if ( pid!=0 ) {
    puts ("Soy el padre, pid=%d\n", getpid());
    // ... otras acciones del padre
    int status;
    pid_t hijo = wait(&status); // espera por el hijo
    // ... el padre puede continuar
}
else {
    printf("Soy el hijo, pid=%d\n", getpid());
    // ... otras acciones del hijo
    exit(1234);
}
```

Comunicación entre procesos (IPC)

- ¿Cómo pueden intercambiar datos varios procesos? Dos modelos:
 - Memoria compartida
 - Paso de mensajes
- Mecanismos clásicos (Unix, Windows):
 - Usar un fichero para compartir datos (ineficiente)
 - Zonas de memoria compartida
 - Tuberías (*pipes*)
 - Sockets
 - Llamadas a procedimiento remoto (RPC)

HILOS (THREADS)

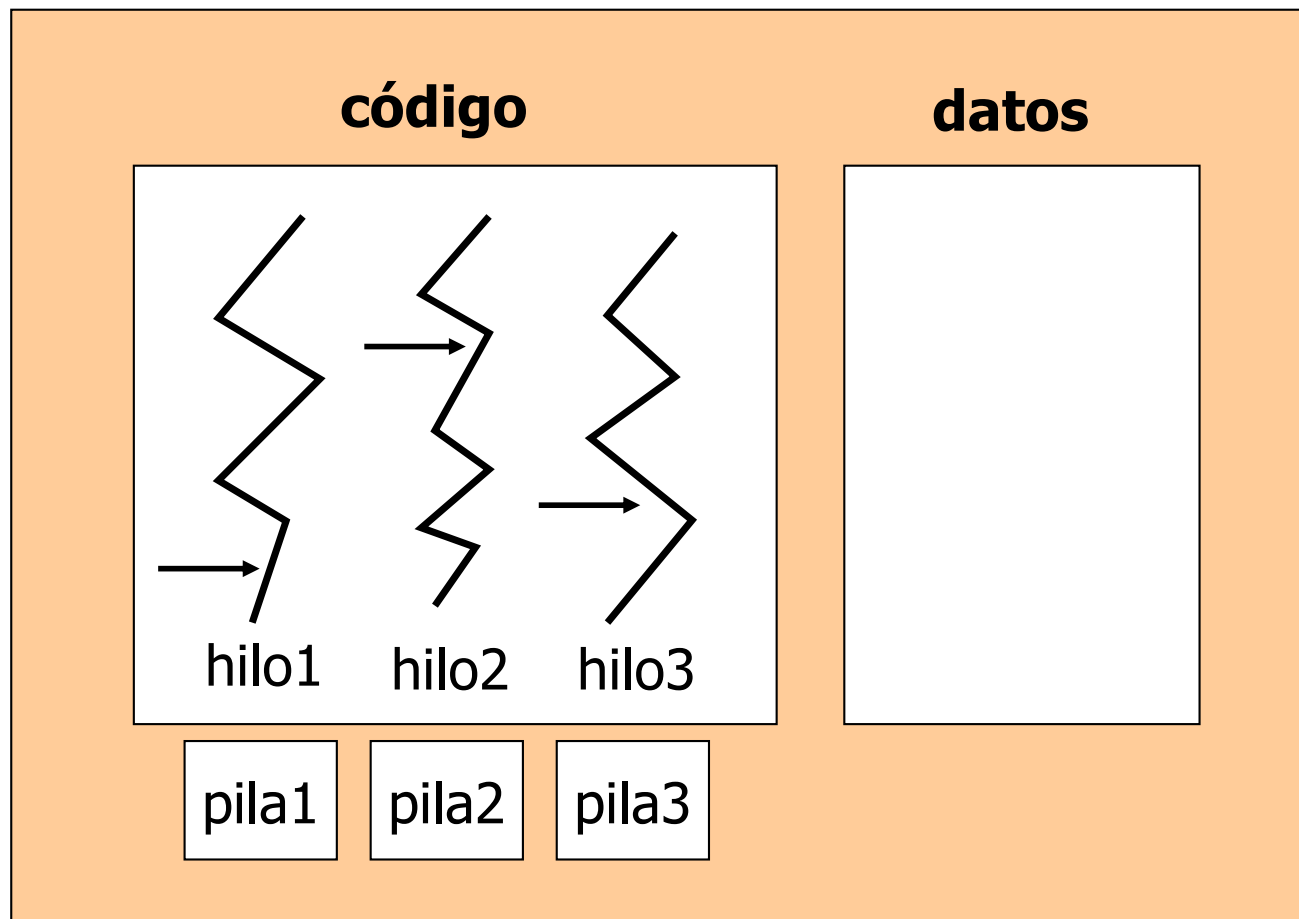
Hilos (*threads*)

- Un **hilo**, **hebra** o **proceso ligero** es una unidad básica de ejecución, con su propio:
 - contador de programa
 - registros de CPU
 - pila (stack) para los datos locales
- Los hilos dentro de una misma aplicación comparten:
 - código
 - datos (variables globales)
 - recursos del S.O. (ficheros, E/S, etc.)
- NOTA: *thread* se pronuncia «**zred**», NO «zrid»

Procesos pesados, procesos ligeros

- Un **proceso pesado** contiene uno o varios hilos que comparten la misma memoria y recursos.
 - Proceso pesado = aplicación
 - Hilos = actividades concurrentes dentro de la aplicación
- Hilos = **procesos ligeros** (*lightweight processes*)

La memoria en una aplicación multihilo



Proceso pesado

Beneficios de los hilos

- Ventajas de N hilos en un único proceso pesado, respecto a N procesos pesados:
 - Comunicación más sencilla entre los procesos (memoria compartida)
 - Más concurrencia con menos recursos (usan el mismo espacio de memoria para código y datos)
 - La conmutación entre hilos puede ser más eficiente (ej. al cambiar de hilo no hay que limpiar cachés y tablas de páginas)

API de hilos: pthreads (Unix)

se corresponde directamente
con servicios del sistema operativo

```
#include <pthread.h>

void* rutina1 ( void* arg ) { /* acciones que realizará un hilo */ }
void* rutina2 ( void* arg ) { /* acciones que realizará otro hilo */ }

int main()
{
    // Declara dos objetos de tipo hilo
    pthread_t un_hilo, otro_hilo;

    // Lanza un hilo concurrente que ejecuta: rutina1(NULL)
    pthread_create ( &un_hilo, NULL, rutina1, NULL);

    // Lanza otro hilo que ejecuta: rutina2(NULL)
    pthread_create ( &otro_hilo, NULL, rutina2, NULL);

    // El hilo principal espera a que finalicen los otros dos hilos
    void* dummy; // no se usa en este ejemplo
    pthread_join ( un_hilo, &dummy );
    pthread_join ( otro_hilo, &dummy );
    exit(0);
}
```

API de hilos: Java

interfaz de alto nivel: el compilador la traduce
a llamadas a la máquina virtual Java

```
// Método uno: implementar la interfaz Runnable
public class UnTipoDeHilo implements Runnable {
    public void run() { /* acciones del hilo */ }
}

// Método dos: heredar de la clase Thread
class OtroTipoDeHilo extends Thread {
    public void run() { /* acciones del hilo */ }
}

public static void main(String args[]) {
    Thread t1 = new Thread(new UnTipoDeHilo);
    t1.start();

    Thread t2 = new OtroTipoDeHilo();
    t2.start();

    // espera a que acaben los hilos
    t1.join();
    t2.join();
}
```

Fundamentos de los Sistemas Operativos

Tema 2. Procesos FIN DEL PRIMER BLOQUE

© 1998-2024 José Miguel Santos - Alexis Quesada - Francisco Santana